

03 — Features (and the technology behind each)

Every feature below lists **what it does**, the **tech** it uses, **how it works**, and its **fallback** (the pure-Python path used when a heavy dependency is missing). File references point you to [04_file_by_file.md](#).

A. The ranking engine

A1. Streaming candidate loader

- **What:** reads the 100K-line `candidates.jsonl` without loading it all into RAM.
- **Tech:** Python generators, `json`, `hashlib` (SHA-256 of the file for cache keys).
- **How:** `iter_candidates()` yields one parsed profile at a time; `corpus_text()` flattens a profile into one searchable string; `rerank_text()` builds a shorter, focused string (title + recent role + top skills) for the cross-encoder.
- **Files:** `pipeline/loader.py`.

A2. Feature engineering — 40 numeric features

- **What:** turns each messy profile into a fixed vector of 40 numbers — the single contract every model in the system agrees on.
- **Tech:** `pandas`, `NumPy`; date math for tenure/recency.
- **How:** `build_feature_row()` computes things like years of experience, average tenure, JD-skill match count, retrieval-skill count, assessment scores, product-company ratio, behavioural composite, and location fit. The exact list lives in `core/constants.NUMERIC_FEATURE_COLUMNS`. Because it's a fixed schema, you can swap the model behind it without touching anything else.
- **Files:** `pipeline/feature_engineering.py`, `core/constants.py`.

A3. Honeypot detection

- **What:** catches the ~80 fake “trap” profiles (impossible career arithmetic).
- **Tech:** rule-based, vectorised `pandas` — chosen deliberately over ML for **high precision** (we must not flag real people).
- **How:** a few strict rules (e.g. total role durations exceed the person's age, or experience that can't fit a plausible timeline). Honeypots are *kept* in the matrix but flagged, so the API can explain the exclusion; the scorer pins their score to 0 so they can never enter the top 100. On the real data this flags ~65.
- **Files:** `pipeline/honeypot_detector.py`.

A4. JD deconstruction

- **What:** turns the free-text job description into a structured `JobSpec` (required skills, seniority, location preference, must-haves).
- **Tech:** **Gemini** (`google-generativeai`) *optionally*, offline only.
- **How:** if an API key is set, an LLM extracts structured requirements; otherwise a keyword/rule parser does it. This is the **only** place an LLM may run, and only in Phase 1 — so the graded Phase 2 never needs the network.
- **Files:** `pipeline/jd_deconstructor.py`.

A5. Gate filter

- **What:** removes candidates who fail hard requirements before ranking.
- **Tech:** vectorised `pandas`; no LLM, no network.

- **How:** `apply_gates()` checks the JobSpec requirements against the feature matrix (e.g. “all-consulting career”, keyword-stuffer, inactive/unresponsive). Gated rows are flagged and written to `excluded_candidates.csv` with a reason.
- **Files:** `pipeline/gate_filter.py`.

A6. Semantic embeddings (dense retrieval)

- **What:** represents each profile and the JD as a 384-dimension vector so that *meaning*, not just words, can be compared.
- **Tech:** **sentence-transformers** (`all-MiniLM-L6-v2`).
- **How:** `embed_corpus()` encodes every profile; the JD is encoded the same way; cosine similarity ranks profiles by semantic closeness to the JD.
- **Fallback:** a **NumPy feature-hashing embedder** — hashes tokens into buckets with signs and L2-normalises. Deterministic, offline, no torch. Coarser, but the rest of the pipeline can’t tell the difference (same 384 dims).
- **Files:** `pipeline/embedder.py`.

A7. BM25 keyword index (sparse retrieval)

- **What:** classic keyword relevance — which profiles literally contain the JD’s important terms, weighted by rarity.
- **Tech:** **rank-bm25** (Okapi BM25).
- **How:** builds an index over all profile texts, scores the JD query against it.
- **Fallback:** a hand-written **two-pass streaming inverted index** with document-frequency pruning — built to keep memory low on 100K docs (this is what fixed an out-of-memory problem; see [04](#)).
- **Files:** `pipeline/bm25_indexer.py`.

A8. Hybrid retrieval (Reciprocal Rank Fusion)

- **What:** combines keyword (BM25) and semantic (embedding) rankings into one.
- **Tech:** **Reciprocal Rank Fusion (RRF)** — a simple, robust rank-combining formula: $\text{score} = \sum 1/(k + \text{rank_in_each_list})$.
- **How:** each method ranks the candidates; RRF blends the *ranks* (not the raw scores, which are on different scales). The result is the `retrieval_score`.
- **Files:** `pipeline/retrieval.py`.

A9. ML fit scorer

- **What:** a machine-learning model that predicts how well a candidate fits.
- **Tech:** **XGBoost** (`XGBRanker`, an NDCG-objective LambdaMART). Falls back to **LightGBM**, then **scikit-learn** `HistGradientBoostingRegressor`, then a hand-tuned linear **formula**.
- **How:** the model is trained in Phase 1 (see A11) and saved with **joblib**; `predict_fit()` loads it and produces a normalised `ml_fit` score per candidate. If the model file is missing or won’t load, it returns the formula directly — the ranker still works.
- **Files:** `ml/trainer.py`, `ml/predictor.py`.

B. The accuracy layer (sharpening the top of the list)

B1. Graded pseudo-labels → LambdaMART

- **What:** the dataset has no “hire/reject” labels, so we *manufacture* graded relevance labels (tier 0–5) and train a learning-to-rank model to optimise the contest metric directly.
- **Tech:** percentile bucketing (`pandas/NumPy`); optionally **Gemini** to grade a sample; **XGBRanker** with `objective="rank:ndcg"` (LambdaMART).

- **How:** a richer-than-proxy composite is bucketed into tiers (a few “5”s, more “3”s, lots of “0”s — like real relevance). If a key is set, Gemini grades the top ~1,500 against the JD and overrides the heuristic for that slice. The ranker then learns the *actual* definition of relevance, not a proxy.
- **Fallback:** heuristic tiers (no LLM) and the regressor/formula (no XGBoost).
- **Files:** ml/pseudo_labels.py, ml/trainer.py.

B2. Cross-encoder reranker

- **What:** re-scores the *top* ~300 candidates by reading the JD and a profile **together** in one transformer pass — far more precise than scoring them apart.
- **Tech:** **sentence-transformers** CrossEncoder (ms-marco-MiniLM-L-6-v2).
- **How:** only the head is reranked (that’s where NDCG@10/@50 is decided); reranking all 100K would be too slow. The result is cached as rerank_score.
- **Fallback:** a **lexical** BM25-flavoured overlap of JD terms with each profile — pure NumPy, still a real signal.
- **Files:** ml/reranker.py.

B3. Bradley-Terry tournament (our differentiator)

- **What:** for the very top (~60), runs an actual **round-robin tournament** — “between these two, who’s better?” — and resolves it into a globally consistent ranking.
- **Tech:** **NumPy**; Bradley-Terry maximum-likelihood via MM iteration (Hunter, 2004).
- **How:** every candidate “plays” every other on several JD-relevant criteria; each match is a soft win-share; the round-robin (which can contain cycles like A>B>C>A) is resolved by the Bradley-Terry model into a single strength per candidate. Because only *ordinal* comparisons matter, it’s immune to feature-scale quirks. Cached as tournament_score.
- **Files:** pipeline/tournament.py.

B4. LLM head-scoring (the smartest signal)

- **What:** an LLM reads each *top* ~250 candidate against the JD and scores fit 0-100 — judgment the GBDT can’t do (it reads the career narrative, not just numbers).
- **Tech:** GLM / Ollama / Gemini via core/llm.py; runs **offline in precompute**, batched (~20 candidates per call), cached as llm_fit_score.
- **How:** only the head is scored (where NDCG@10/@50 is decided); the result is blended into the head like the rerank/tournament signals. rank.py reads the cached column, so it stays no-network/<5min.
- **Fallback:** off by default (LLM_HEAD_SCORING_ENABLED=true to enable). With no LLM reachable it scores nothing and the scorer adds nothing — baseline unchanged.
- **Files:** ml/llm_scorer.py, pipeline/scorer.py, precompute.py.

B5. Evaluation harness

- **What:** computes the exact contest metric so every change is measured.
- **Tech:** pure NumPy.
- **How:** implements NDCG@k, MAP, P@k and the weighted composite. We score against the graded pseudo-labels (a stand-in for the hidden ground truth) to A/B any change and confirm the head actually improved.
- **Files:** ml/evaluate.py.

B6. Final score merge

- **What:** combines every signal into one number per candidate.
- **Tech:** pandas/NumPy.
- **How:** core = $0.40 \cdot \text{ml_fit} + 0.35 \cdot \text{retrieval} + 0.25 \cdot \text{behavioural}$; the rerank and tournament signals fold into the head as a **convex blend** (so scores stay in [0,1] and don’t saturate);

then $\times$ location_factor $\times$ signal_modifier + evidence_bonus. Honeypots/gated $\rightarrow$ 0. When the extra columns are absent, the formula is exactly the original — so the add-ons are invisible by default.

- **Files:** pipeline/scorer.py, pipeline/signal_scorer.py.
-

C. Output

C1. Reasoning generator

- **What:** the one-sentence human justification on each shortlisted candidate.
- **Tech:** template-based prose (no LLM needed at rank time).
- **How:** picks the most salient, JD-relevant facts (seniority, proven skills, product-company experience, activity) and writes a varied, readable sentence.
- **Files:** output/reasoning_generator.py.

C2. Submission writer + validator

- **What:** writes submission.csv and checks it against the official rules.
- **Tech:** pandas; the organiser's validate_submission.py.
- **How:** assembles exactly 100 rows (candidate_id, rank, score, reasoning), writes the CSV, then runs the validator to guarantee it passes before you submit.
- **Files:** output/submission_writer.py, validate_submission.py.

C3. Coverage loop agent

- **What:** a lightweight “are we missing anything?” check over the shortlist.
 - **Tech:** pure Python (no LLM, no network).
 - **How:** a plan→retrieve→reflect loop reports coverage across dimensions (retrieval, LLM experience, production evidence, availability). It's informational — it does not change the final order.
 - **Files:** pipeline/loop_agent.py.
-

D. The production service

D1. FastAPI application

- **What:** the HTTP API around the engine.
- **Tech:** **FastAPI**, **uvicorn/gunicorn**, **Pydantic v2**, **sse-starlette**.
- **How:** an app factory wires routers (auth, rank, developers, search, admin, health), middleware, error handlers, metrics and tracing. Validation happens at the edge with Pydantic schemas.
- **Files:** api/ (app.py, main.py, deps.py, schemas.py, routers/*).

D2. Authentication, API keys & RBAC

- **What:** logins, machine keys, and role-based permissions.
- **Tech:** **PyJWT** (JWT access/refresh tokens), **bcrypt** (password hashing), per-user API keys.
- **How:** password login issues JWTs; users mint API keys (shown once, stored hashed). A dependency authenticates either a key or a JWT and enforces roles (admin / recruiter / viewer).
- **Files:** core/security.py, api/deps.py, api/routers/auth.py.

D3. Database & repositories

- **What:** persistent storage for users, jobs, results, audit logs, and the live layer's developers/evidence.

- **Tech:** **SQLAlchemy 2.0 async** + **asyncpg** (Postgres) / **aiosqlite** (dev); **Alembic** migrations.
- **How:** ORM models use UUID string PKs and portable JSON columns so the *same* schema runs on Postgres and SQLite. All DB access goes through a **repository layer**, so routers/services never write raw queries.
- **Files:** db/models.py, db/repositories.py, db/base.py, alembic/.

D4. Caching & rate limiting

- **What:** caches expensive results; throttles abusive callers.
- **Tech:** **Redis** with an in-memory fallback.
- **How:** a Cache abstraction supports read-through caching and eviction (LRU/ LFU). A fixed-window rate limiter uses Redis INCR (or in-memory counters).
- **Files:** services/cache.py, services/ratelimit.py.

D5. Job queue & orchestration

- **What:** ranking runs as a background job; the request returns a `job_id` immediately.
- **Tech:** an in-process **asyncio PriorityQueue** (dev) or **Redis Streams** with consumer groups (prod); **SSE** for live progress.
- **How:** `JobManager.submit()` records the job, commits, enqueues a message, and returns. A `QueueWorker` runs the engine in a thread. Delivery is *at-least-once* with retries and a dead-letter queue; the consumer is *idempotent*.
- **Files:** services/jobs.py, services/queue.py, services/ranker.py.

D6. Observability & resilience

- **What:** metrics, logs, traces, and failure isolation.
 - **Tech:** **prometheus-client**, **structlog** (JSON), **OpenTelemetry** (→ Jaeger), a custom async **circuit breaker**.
 - **How:** middleware records request metrics and a request-id on every log line; spans trace requests when tracing is enabled; the circuit breaker stops hammering a failing dependency (e.g. the GitHub API) and recovers automatically.
 - **Files:** api/metrics.py, core/logging.py, core/tracing.py, core/circuit_breaker.py, api/middleware.py.
-

E. The live signal ingestion layer

E1. GitHub ingestion

- **What:** pulls a developer's real work (commits, PR reviews, issues) from GitHub.
- **Tech:** **GitHub GraphQL API**, async **httpx**, the circuit breaker.
- **How:** an async client fetches repos and contributions within rate limits; the sync core is shared by the API (await directly) and Celery (background).
- **Fallback:** without a token, ingestion simply no-ops.
- **Files:** ingestion/github_client.py, ingestion/sync.py, ingestion/tasks.py.

E2. Evidence confidence scoring

- **What:** how much to trust a skill claim, based on the evidence behind it.
- **Tech:** a transparent weighted formula (NumPy/Python).
- **How:** $\text{confidence} = \text{clamp}(\sum \text{source_weight} \cdot (\text{complexity}/5) \cdot 2^{(-\text{age}/365)} / \text{normaliser}, 0, 1)$. A merged PR (`commit_diff`, weight 1.0) is worth more than a resume claim (0.2); evidence **decays** with a one-year half-life. So fresh, hard evidence outranks stale, soft claims.
- **Files:** intelligence/confidence_scorer.py.

E3. Skill extraction & artifact summaries

- **What:** map messy skill strings to canonical names; summarise each work item.
- **Tech:** **spaCy** (optional) for skill extraction; **Gemini** (optional) for summaries.
- **How:** skills are normalised to a taxonomy; each artifact (commit, PR, issue) gets a short natural-language summary. Both fall back to deterministic heuristics with no model.
- **Files:** `intelligence/skill_extractor.py`, `intelligence/summariser.py`.

E4. Proof-of-work knowledge graph + hybrid (graph + vector) query

- **What:** stores evidence in a graph and lets a recruiter query it in English, fusing exact graph traversal with semantic search for better recall.
- **Tech:** **Neo4j** (with an in-memory graph fallback); SHA-256 anchoring; embeddings (sentence-transformers or the NumPy hashing fallback); RRF fusion.
- **How:** developers → skills → evidence are nodes/edges; each evidence node is content-hashed so it's *verifiable* (tamper-evident). A query runs **two** retrievers and fuses them by rank (RRF, like the main ranker):
 1. **structured skill traversal** — precise: developers tagged with the canonical skill (e.g. the 3-hop “production debuggers” query);
 2. **semantic vector search** over evidence summaries — recall: catches paraphrases the skill tag misses (“real-time event streaming” → a “Kafka” developer whose commit says “streaming telemetry consumer”). Confidence **accumulates** across evidence; each result is tagged `match_via` (skill / semantic / both).
- **Measured:** on a gold-set A/B, hybrid lifts `recall@5` from 0.6 → 1.0 and `MRR` 0.6 → 1.0 vs skill-only (it answers the paraphrased queries that returned nothing before), trading some `precision@k` — the usual recall/precision balance.
- **Why not “full Graph RAG”:** the LLM is used to *parse* the query and to *summarise* evidence at ingestion, not to generate the final answer at query time — so this is graph-RAG-style retrieval, not generative Graph RAG.
- **Files:** `graph/schema.py`, `graph/queries.py`, `graph/vector_index.py`, `graph/natural_language_query.py`, `graph/evaluate.py`.

E5. Composition with the static ranker

- **What:** verified developers float slightly above identical static-only twins.
 - **How:** the graph contributes an additive `evidence_confidence_bonus` column that the scorer adds at the end. It defaults to 0, so the graded submission is **unchanged** when the live layer is off. It augments, never overrides.
 - **Files:** `pipeline/feature_engineering.py` (`apply_live_signals`), `pipeline/scorer.py`.
-

F. Frontend (recruiter UI)

- **What:** a web console to run rankings, watch progress, browse results, and search the evidence graph.
 - **Tech:** **React 18 + TypeScript, Vite, Tailwind CSS, TanStack React Query, react-router-dom, Recharts, lucide-react.**
 - **How:** pages for Login, Dashboard, Ranking Results, Natural-Language Search, and Settings; a Server-Sent-Events hook streams job progress; an evidence timeline and proof-of-work card visualise the live layer.
 - **Files:** `frontend/src/**`.
-

G. Performance layer (speed, no feature loss)

- **Device auto-detect** — runs the transformers on Apple **MPS** / **CUDA** when available, else CPU. `core/device.py`.
- **ONNX + int8 toggle** — optional ONNX Runtime backend with quantised weights (~2-4× on CPU), with onnx→torch fallback. `core/device.py`.
- **Leaner LambdaMART** — trains on all positives + a sampled set of negatives, histogram tree method, early stopping. `ml/trainer.py`.
- **Focused rerank text** — the cross-encoder reads a tight projection, not the full blob. `pipeline/loader.rerank_text`.
- **Content-hash cache** — `--resume` reuses embeddings / model / rerank scores when their inputs are unchanged; change only the JD and the 100K embeddings are reused. `core/artifact_cache.py`.

Full detail in [./ACCURACY.md](#). Next: [04_file_by_file.md](#).